



Tecnológico de Monterrey

Campus Querétaro

Evidencia Demostración de un Paradigma de Programación

Inaki Mancera Llano - A01708827

Materia: Implementacion de Metodos Computacionales

Fecha de entrega
22 de Mayo 2026

1. Complejidad temporal de la solución aplicada (Funcional).

La solución principal fue implementada en DrRacket usando el paradigma de programación funcional. El problema consiste en analizar una cadena de piedras representadas por los caracteres R, G y B, y contar cuántas piedras deben retirarse para que no existan dos piedras vecinas del mismo color.

La función principal de la solución es `removedCount`, la cual recibe una lista de caracteres y compara recursivamente cada piedra con la siguiente. Si ambas piedras son iguales, se suma 1 al resultado; si son diferentes, no se suma nada y la función continúa con el resto de la lista.

Sea n el número de piedras en la cadena de entrada. La función revisa cada elemento de la lista una sola vez, comparándolo con su vecino inmediato. Por esta razón, la complejidad temporal de la solución es: **$O(n)$** . Esto significa que el tiempo de ejecución crece de forma lineal con respecto al tamaño de la entrada. Si la cadena tiene más piedras, la cantidad de comparaciones aumenta proporcionalmente.

2. Solución alternativa (scripting con expresiones regulares).

Como paradigma alternativo, el problema también podría resolverse usando scripting con expresiones regulares, por ejemplo en un lenguaje como Python. Este enfoque no se implementa como solución principal, pero sirve para comparar cómo el mismo problema puede abordarse desde otro paradigma.

Para este problema, en lugar de comparar recursivamente cada piedra con su vecina, la solución alternativa buscaría grupos consecutivos de piedras del mismo color.

Cada grupo representa piedras iguales que están juntas. Para que no haya dos piedras iguales juntas de cada grupo solo puede quedar una. Por ello la cantidad de piedras que deben retirarse por grupo se calcula con la fórmula **longitud de grupo - 1**.

Una expresión regular que podría usarse para detectar estos grupos es: **`([RGB])\1*`**. Esta expresión busca uno de los tres caracteres, los deja como grupo si los encuentra juntos y que hayan 0 o más repeticiones del mismo carácter.

En este paradigma, el problema se interpreta como una tarea de reconocimiento de patrones dentro de una cadena. En lugar de resolverlo mediante recursión, se describe qué patrón se quiere encontrar y después se procesa cada coincidencia encontrada.

La complejidad temporal de esta solución también sería: **$O(n)$** . Esto se debe a que la expresión regular debe recorrer la cadena de entrada para detectar los grupos consecutivos.

3. Comparación de ambas soluciones.

La solución funcional en DrRacket interpreta el problema como una proceso recursivo de los datos. Primero se convierte el string en una lista de caracteres, y después se procesa esa lista comparando cada elemento con el siguiente. La solución está basada en funciones, recursión y ausencia de estado mutable, lo cual corresponde al paradigma funcional.

Por otro lado, la solución con scripting y expresiones regulares interpreta el problema como una búsqueda de patrones dentro de un string. En vez de hacer explícitamente la comparación piedra por piedra, se detectan directamente los grupos consecutivos de colores iguales y se calcula cuántas piedras sobran en cada grupo.

Característica	Solución Funcional en DrRacket	Solución alternativa con scripting
Paradigma	Programación Funcional	Scripting con expresiones regulares
Lenguaje	DrRacket	Python
Idea Principal	Comparar recursivamente cada piedra con la siguiente	Detectar grupos consecutivos de piedras
Estructura de Datos	Lista de caracteres	String
Técnica	Recursión y funciones	Identificación de patrones
Estado Mutable	No se requiere	Puede requerir acumulador
Complejidad Temporal	$O(n)$	$O(n)$

Ambas soluciones tienen complejidad temporal lineal $O(n)$ porque ambas deben analizar la cadena de texto completa. La diferencia principal no está en la eficiencia temporal, sino en la forma de modelar el problema. La solución funcional fue elegida como implementación principal porque permite demostrar de forma clara el proceso recursivo y la composición de las funciones.